DWA and Nav2's DWB Controller, Start to Finish

Contents

1. Overview . . . 2
2. What goes in and what comes out . . . 5
3. Which commands the robot can reach: the dynamic window . . . 6
   - 3.1 One point per command . . . 6
   - 3.2 Reachable in one cycle: Fox's dynamic window . . . 7
   - 3.3 Safe to stop: admissible velocities . . . 9
   - 3.4 DWB's window and its 375 samples . . . 10
   - 3.5 Two settings that fail on our robot . . . 11
4. Where each command goes: the rollout . . . 12
   - 4.1 One rollout, step by step . . . 12
   - 4.2 Two generators, two kinds of rollout . . . 14
5. Which command is best: the critics . . . 15
   - 5.1 The costmap the critics read . . . 15
   - 5.2 The obstacle critic: illegal or a small cost . . . 17
   - 5.3 Following the plan: the distance grids . . . 17
   - 5.4 Two critics for the goal and for oscillation . . . 19
   - 5.5 The total, and the lowest one . . . 20
6. One cycle start to finish, in the order the code runs . . . 22
7. The whole drive: DWB against TEB . . . 23
   - 7.1 The plan made once: stopped in front of the person . . . 23
   - 7.2 The plan remade every second . . . 25
   - 7.3 Why DWB is slower than TEB here . . . 25
8. Real systems: DWA in ROS 1 and DWB in Nav2 . . . 26
9. Summary . . . 28
10. Sources . . . 29
11. Key terms . . . 30

Where this fits:

- **Builds on:** Differential-drive kinematics; Inevitable collision states; Motion primitives and lattice search; Global plan and local planner.
- **Leads to:** Controller server; DWA, TEB and MPC compared.

1. Overview

Key point: The dynamic window approach chooses each command by trying every speed and turn rate the robot can reach, imagining each one held for 1.7 s, and keeping the one with the lowest weighted score. Nav2's DWB controller does this 20 times a second. In our Python version of DWB on the room scene, the robot brakes for the person in the cycle they appear, gets round only when the global plan is remade, and is accepted at the door at 13.3 s, against 8.0 s for RO-058's timed elastic band.

The robot can already reach the door round a person the map does not have. RO-058's local planner (G-2911), a timed elastic band, solves an optimisation problem every 0.1 s and stops on the door pose at 8.0 s, against 6.7 s in a clear room. Its numerical solver needs a good first guess, and a poor one leaves the band on the wrong side of the person.

ROS's navigation software has long shipped a simpler local planner: the dynamic window approach, DWA (Fox et al. 1997), in ROS 1, and its successor in Nav2, the DWB controller, which Nav2's documentation calls its default controller (Nav2 docs: DWB configuration; Macenski et al. 2023 §III-A). DWA solves no optimisation problem. This Note runs it on the same scene:

- the robot starts at rest at (2.0, 1.0) facing 30° and must stop at the door pose (3.6, 3.5) facing 90°;
- it is a disk of radius 0.25 m; each wheel turns at most 0.6 m/s, and its forward speed changes by at most 0.5 m/s^2;
- the global plan is RO-034's A* route on 0.5 m cells, 3.698 m from the robot to the door pose (RO-058's 3.414 m is measured between cell centres);
- at 2.4 s a person, a disk of radius 0.2 m, steps onto the plan at (2.90, 2.75).

At 2.4 s the robot drives at 0.568 m/s, and 0.578 m separate its edge from the person's. Within one cycle of 0.05 s, the controller must send one speed and one turn rate. The decision has four parts, and the Note takes them in turn:

1. **reach:** which commands the robot can reach from 0.568 m/s (Section 3);
2. **where:** where each command would take it (Section 4);
3. **best:** which one is best (Section 5);
4. **door:** whether repeating this 20 times a second gets the robot to the door, and how it compares with TEB's 8.0 s (Sections 6 and 7).

What the runs are. Every run in this Note is our own Python version of DWB's algorithm, written from the navigation2 code (branch `main`, read 2026-10-12) with its parameter values. Nav2's software itself is not running. What we simplified:

- the local costmap, the grid of costs round the robot that DWB reads, is built from the

room's exact shapes, and a cell's cost from the exact distance from its centre to the nearest obstacle, not from cells a laser marks;
- the robot's pose and speed are measured without error;
- the global plan is RO-034's A* on 0.5 m cells, not Nav2's planner, made once or remade once a second;
- the critics, DWB's scoring functions, are the seven of the Nav2 configuration guide's DWB example;
- every cycle finishes within its 0.05 s, and no recovery behaviour, such as backing up or spinning, runs after an abort;
- the robot follows each command within 0.5 m/s^2 for speed and 3.2 rad/s^2 for turn rate, with no velocity smoother, the filter Nav2 can place between the controller and the wheels.

Figure 1 plays the main run. Watch the orange rollout, the command DWB chose in that cycle imagined 1.7 s ahead: it shortens when the person appears at 2.4 s, bends left after the new plan of 3.0 s, and ends in a turn on the spot.

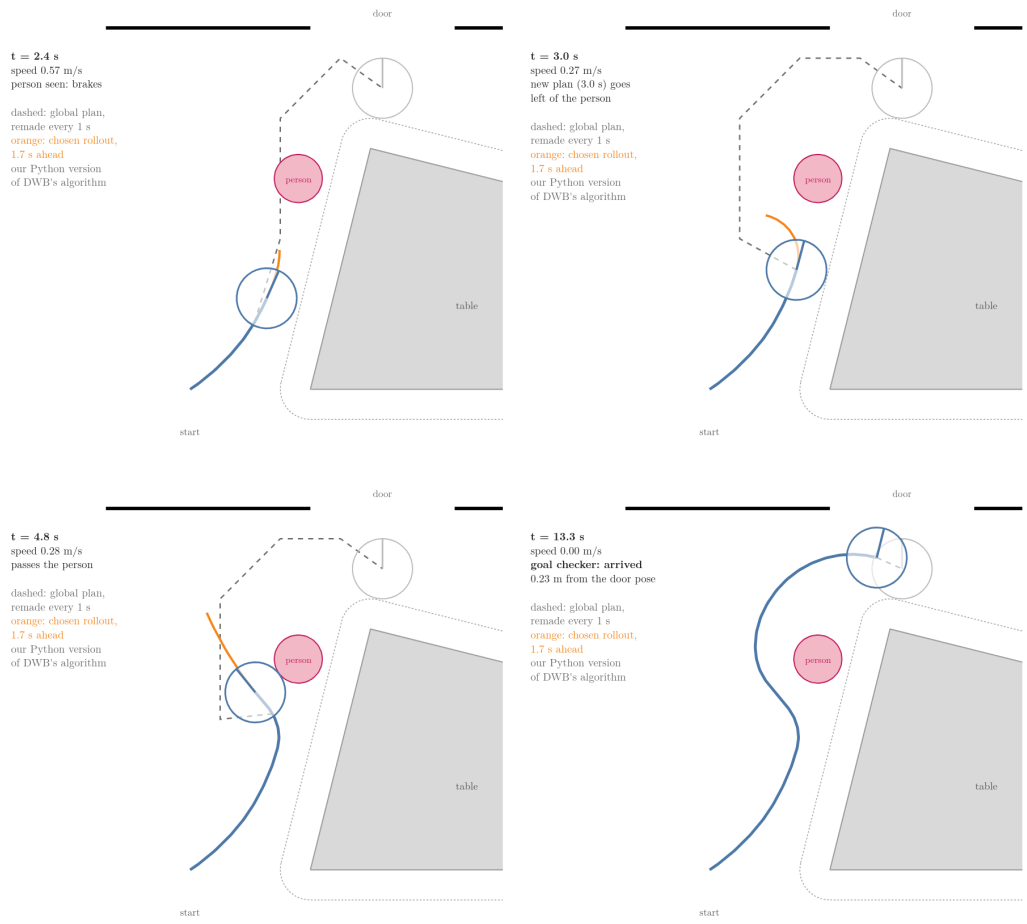


Figure 1: Our Python version of DWB's algorithm (written from the navigation2 code, not Nav2 running) drives the room, one frame per 0.2 s. Grey dashed: the global plan, A* on 0.5 m cells, remade every second. Orange: the rollout DWB chose in this cycle, 1.7 s ahead. Blue: the robot and where it has driven; pale circle: the door pose. At 2.4 s a person (pink) steps onto the plan and the robot brakes; the plan made at 3.0 s goes left of them; the robot passes them, turns on the spot from 9.7 s, and the goal checker accepts it at 13.3 s, 0.23 m from the door pose

The robot never touches the person but passes 0.002 m from them, and the goal checker, Nav2's test for arrival, accepts it 0.23 m from the door pose at 13.3 s (notebook). The sections build this, in the order the code runs it:

- what goes in and what comes out (Section 2);
- the window of reachable commands, and the 375 samples taken from it (Section 3);
- the rollout of each sample (Section 4);
- the costmap and the critics that score each rollout (Section 5);
- one whole cycle through Nav2's controller server (Section 6);
- the drive to the door, with the plan made once and remade every second (Section 7);
- DWA in ROS 1 and DWB in Nav2 (Section 8).

2. What goes in and what comes out

> **Key point:** Nav2 calls a local planner a controller: 20 times a second it takes the global plan, the local costmap and the measured speed, and sends one speed and one turn rate to the wheels. DWB is one such controller.

The whole problem of the opening is one cycle of a loop, so we first set out what the loop reads and what it writes.

A local planner that Nav2 calls a controller. A feedback (G-2941) loop measures the robot and sends a command, again and again. The local planner of RO-058 is used exactly this way: it runs every cycle, from where the robot is, and the robot drives what it sends. Nav2's documentation therefore says: "Controllers, also known as local planners in ROS 1, are the way we follow the globally computed path or complete a local task" (Nav2 docs: navigation servers). The program that runs the controller is Nav2's controller server, taught with the rest of Nav2 in Nav2 start to finish.

The inputs and the output. Figure 2 shows them, with their rates on `main` (controller_server parameters; nav2_bringup `nav2_params.yaml`):

- **the global plan:** poses 5 cm apart from the robot to the goal; the default behaviour tree of Nav2's released versions (Humble, Jazzy, Kilted) remakes it once a second, while the tree on `main` remakes it only when the goal changes, more than 4.0 m of path remain or the rest of the path is blocked (Nav2's default tree);
- **the local costmap:** a grid of 5 cm cells, 3 m by 3 m round the robot, updated 5 times a second, in which every cell holds a cost for being there (Section 5.1);
- **odometry:** the robot's measured speed and turn rate;
- **out:** one speed and one turn rate, on the `cmd_vel` topic, 20 times a second (`controller_frequency` 20.0).

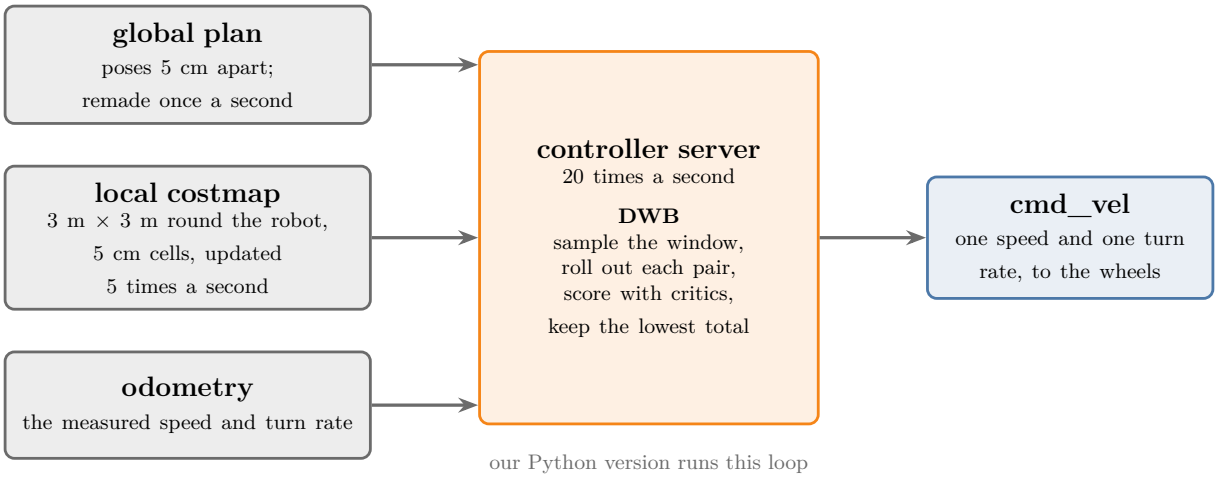


Figure 2: What goes into Nav2's controller server with DWB and what comes out: the global plan (poses 5 cm apart, remade once a second), the local costmap (3 m by 3 m round the robot, 5 cm cells, updated 5 times a second) and odometry (the measured speed and turn rate) go in; 20 times a second DWB samples the window, rolls out each pair, scores the rollouts with critics and keeps the lowest total; one speed and one turn rate go out on cmd_vel

DWB. The controller that chooses each command by sampling commands within the robot's acceleration limits, simulating each one forward and scoring the results with a list of plug-in scoring functions is the **DWB controller** (G-3255) of Nav2, in the package `nav2_dwb_controller` (DWB README). Its name records its origin: "DWB as in the 'next' after DWA", the dynamic window approach of ROS 1 (Macenski et al. 2023, footnote 3). The loop is fixed; what DWB does inside one cycle starts with the commands it may choose from.

3. Which commands the robot can reach: the dynamic window

> **Key point:** The first part of the opening problem, reach: from 0.568 m/s, only some speeds and turn rates can be reached in time, because speed changes by at most 0.5 m/s^2 and turn rate by at most 3.2 rad/s^2. DWA keeps those reachable in one cycle; DWB keeps those reachable in its 1.7 s look-ahead, and tries 375 of them.

The robot cannot change to an arbitrary command at once: at 2.4 s it drives at 0.568 m/s, and it cannot stop instantly.

3.1 One point per command

The idea. A command is two numbers, a speed and a turn rate. Held fixed, the pair drives the robot along one circular arc, a straight line when the turn rate is 0 (four ways to drive). So we can draw every command as one point of a plane: the turn rate across, the speed up (Fox et al. 1997 §4.1). With:

- v, the **linear velocity** (G-2291), the forward speed, in m/s;
- ω, the **angular velocity** (G-2292), the turn rate, in rad/s, positive to the left.

The point (0.5 rad/s, 0.4 m/s) means: drive 0.4 m/s while turning 0.5 rad/s, an arc of radius 0.8 m. Stage 1 of Figure 3 shows it.

The robot's limits. Not every point is allowed. Each wheel turns at most 0.6 m/s, and the wheel speeds are v plus or minus ω times half the wheel separation (the wheel speeds for a given motion). With:

- L, the wheel separation, 0.2 m;
- v and ω as above.

$$v + \omega L/2 \leq 0.6$$

$$v + 0.1\,\omega \leq 0.6$$

The robot also drives only forwards here, and DWB's turn limit `max_vel_theta` is set to 1.0 rad/s, the configuration example's value and our robot's own limit at 0.5 m/s (RO-058 §3.1). Stage 2 of Figure 3 shades the allowed points.

3.2 Reachable in one cycle: Fox's dynamic window

The problem on numbers. A driver at 50 km/h can choose, for the next second, between about 45 and 55 km/h, not 0: the car cannot change speed faster than that. At 2.4 s the robot drives at 0.568 m/s and turns at 0.263 rad/s (stage 3 of Figure 3). In the next cycle of 0.05 s its speed can change by at most 0.5 m/s^2 times 0.05 s, and its turn rate by at most 3.2 rad/s^2 times 0.05 s. Commands outside that range would be asked for but not delivered.

The standard term. The set of commands the robot can reach from its current speed and turn rate within one time interval, given its acceleration limits, is the **dynamic window** (G-3251) (Fox et al. 1997, eq. 15). The window is centred on the current command and moves with it every cycle, which is why it is called dynamic. With:

- v_a, ω_a, the speed and turn rate now, 0.568 m/s and 0.263 rad/s;
- $\dot{v}_{\max}$, the largest change of speed per second, 0.5 m/s^2;
- $\dot{\omega}_{\max}$, the largest change of turn rate per second, 3.2 rad/s^2;
- Δt, the interval, one controller cycle, 0.05 s.

Our numbers first, then the same in symbols:

$$v \geq 0.568 - 0.5 \times 0.05 = 0.543$$

$$v \leq 0.568 + 0.5 \times 0.05 = 0.593$$

$$\omega \geq 0.263 - 3.2 \times 0.05 = 0.103$$

$$\omega \leq 0.263 + 3.2 \times 0.05 = 0.423$$

$$v_a - \dot{v}_{\max}\Delta t \leq v \leq v_a + \dot{v}_{\max}\Delta t$$

$$\omega_a - \dot{\omega}_{\max}\Delta t \leq \omega \leq \omega_a + \dot{\omega}_{\max}\Delta t$$

Stage 4 of Figure 3 draws this window: a small orange box round the current command. The **dynamic window approach** (G-3250), DWA, chooses each command from this window only, and among the commands there it keeps the best one (Fox et al. 1997 §4).

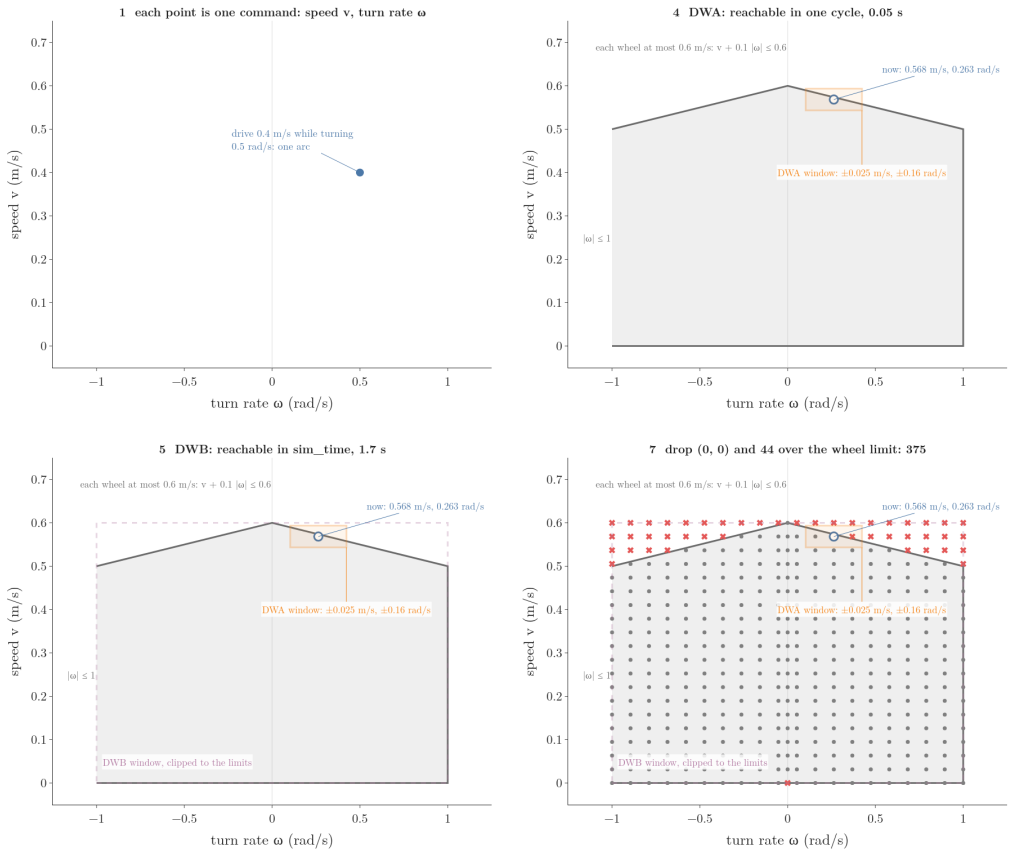


Figure 3: The dynamic window in the plane of speed v (up) and turn rate ω (across) at t = 2.4 s, built up in 7 stages. 1: one point is one command, such as 0.4 m/s while turning 0.5 rad/s, one arc. 2: the robot's limits (grey): forward only, $|\omega|$ at most 1 rad/s, each wheel at most 0.6 m/s. 3: the command now, 0.568 m/s and 0.263 rad/s. 4: DWA's window, reachable in one cycle of 0.05 s: ± 0.025 m/s, ± 0.16 rad/s (orange). 5: DWB's window, reachable in sim_time, 1.7 s, which reaches past every limit and is clipped to them (purple dashed). 6: 20 speeds by 21 turn rates, 420 pairs. 7: (0, 0) and the 44 pairs that ask a wheel for more than 0.6 m/s are dropped (red), 375 are left

3.3 Safe to stop: admissible velocities

A reachable command can still be unsafe: the robot may be unable to stop before the obstacle on its arc. Fox's paper keeps a command only if the robot, braking from it, stops before the nearest obstacle on its arc; such a command is an **admissible velocity** (G-3252) (Fox et al. 1997, eq. 14). The test comes from the stopping distance of planning with motion limits. With:

- d, the free distance along the arc, here the 0.578 m between the robot's edge and the person;
- $\dot{v}_b$, the braking deceleration, 0.5 m/s^2;
- v, the speed, in m/s;
- $v_{\max}$, the largest admissible speed, in m/s.

$$0.568^2/(2 \times 0.5) = 0.323 \text{ m}$$

$$0.323 \leq 0.578$$

$$v_{\max} = \sqrt{2 \times 0.578 \times 0.5}$$

$$= 0.760 \text{ m/s}$$

$$v^2/(2\dot{v}_b) \leq d$$

$$v \leq \sqrt{2\,d\,\dot{v}_b}$$

So at 2.4 s the robot can still stop in front of the person: it needs 0.323 m of the 0.578 m. DWB has no separate admissibility test; it gets the same safety from its rollouts, which last 1.7 s, longer than the 1.2 s the robot needs to stop from 0.6 m/s (notebook), so a braking command is simulated all the way to its stop (Section 4).

3.4 DWB's window and its 375 samples

DWB offers two ways to set the window, its **trajectory generators** (G-3256), the plug-ins that list the commands to try and simulate each one (DWB README):

- `LimitedAccelGenerator`, "similar to DWA used in ROS 1": the window of one cycle, `sim_period`, by default one over `controller_frequency`, 0.05 s;
- `StandardTrajectoryGenerator`, the default, "similar to the trajectory rollout algorithm used in base_local_planner in ROS 1": the window reachable within the whole look-ahead time `sim_time`, 1.7 s.

The default generator's window at 2.4 s (stage 5 of Figure 3), in the code's steps (`one_d_velocity_iterator.hpp`), with the symbols of Section 3.2 and:

- T, the look-ahead time `sim_time`, 1.7 s.

$$v \geq 0.568 - 0.5 \times 1.7 = -0.282$$

$$v \leq 0.568 + 0.5 \times 1.7 = 1.418$$

$$\omega \geq 0.263 - 3.2 \times 1.7 = -5.177$$

$$\omega \leq 0.263 + 3.2 \times 1.7 = 5.703$$

$$v_a - \dot{v}_{\max} T \leq v \leq v_a + \dot{v}_{\max} T$$

$$\omega_a - \dot{\omega}_{\max} T \leq \omega \leq \omega_a + \dot{\omega}_{\max} T$$

Clipped to the limits, the window is every speed from 0 to 0.6 m/s and every turn rate from -1 to 1 rad/s. The generator spreads `vx_samples`, 20 speeds, and `vtheta_samples`, 20 turn rates, evenly across it, and adds a turn rate of exactly 0 because the range crosses zero (`xy_theta_iterator.cpp`):

$$20 \times 21 = 420 \text{ pairs}$$

$$420 - 1 = 419 \text{ without } (0, 0)$$

$$419 - 44 = 375 \text{ within the wheels}$$

The code drops (0, 0) and any pair faster than `max_speed_xy` (`isValidSpeed`); our robot also drops the 44 pairs that ask a wheel for more than 0.6 m/s (stage 7 of Figure 3), such as 0.6 m/s with 0.0526 rad/s:

$$0.6 + 0.0526 \times 0.1 = 0.6053 \text{ m/s}$$

Each of the 375 pairs is a candidate command for the next cycle. The window says which commands are reachable; it does not yet say where each would take the robot.

3.5 Two settings that fail on our robot

Two settings that appear harmless make our robot fail in our Python version (notebook):

- **DWA's one-cycle window from rest.** From standing still the window spans 0 to 0.025 m/s. Held for 1.7 s, the fastest of these moves the robot 0.043 m, less than one 5 cm cell, so the critics, which measure distances in cells, give 136 pairs the same lowest total, 50.4. The first in the list, (0 m/s, 0.0084 rad/s), is kept, the robot never speeds up, and after 10 s it is aborted, 0.01 m from the start (notebook). The default generator's 1.7 s window does not have this problem.
- **Our robot's full turn limit.** Our robot can spin at up to 6 rad/s on the spot. With `max_vel_theta` at 6, the 20 turn-rate samples lie 0.573 rad/s apart, too far apart to finish the final turn at the door: the robot turns back and forth between 60° and 73° and is aborted at 15.5 s, short of the 75.68° the goal checker needs (notebook). The configuration example's 1.0 rad/s keeps the samples 0.105 rad/s apart.

The window's 375 pairs are the candidates; the next step is to see where each one goes.

4. Where each command goes: the rollout

> **Key point:** The second part of the opening problem, where: DWB simulates each of the 375 pairs for 1.7 s from the robot's pose and speed, in small steps, letting the speed change towards the pair at the acceleration limits. Braking to 0.158 m/s takes 18 steps of 0.0944 s and stays clear of the person; keeping 0.568 m/s enters the person's band at step 12.

Each pair from Section 3 is only a pair of numbers; to judge it, we must know where it puts the robot.

4.1 One rollout, step by step

The idea. Imagine sending the pair and watching the robot for the next 1.7 s: it changes its speed towards the pair as fast as it may, then drives at it. Simulating a command forward from the robot's current state to predict the path it would drive, as a list of poses, is a **rollout** (G-3253), also called forward simulation (Zheng 2017 §3.1.2). DWB's look-ahead is `sim_time`, 1.7 s by default (`standard_traj_generator.cpp`). Each step uses the Euler step (G-2299) of RO-001: hold the speed and the heading for one short step, move, repeat.

How many steps. The code cuts the 1.7 s so that the robot moves at most `linear_granularity`, 0.05 m, or turns at most `angular_granularity`, 0.025 rad, per step, at the sampled pair's speed and turn rate (`getTimeSteps`). With:

- T, the look-ahead `sim_time`, 1.7 s;
- v_s, ω_s, the sampled pair, 0.158 m/s and 0.263 rad/s for the chosen one;
- Δs, the largest move per step, 0.05 m;
- $\Delta\theta$, the largest turn per step, 0.025 rad;
- n, the number of steps;
- h, the length of one step, in s.

$$0.158 \times 1.7/0.05 = 5.37$$

$$0.263 \times 1.7/0.025 = 17.9$$

$$n = 18$$

$$h = 1.7/18 = 0.0944 \text{ s}$$

$$n = \lceil \max(v_s T/\Delta s,\ |\omega_s| T/\Delta\theta) \rceil$$

$$h = T/n$$

Here $\lceil\ \rceil$ means rounding up to the next whole number.

The speed in each step. The rollout starts from the measured speed and moves it towards the pair by at most the deceleration times one step (`computeNewVelocity`). With:

- v_a, the speed now, 0.568 m/s;
- $\dot{v}_b$, the braking deceleration, 0.5 m/s^2 (Section 3.3);
- v_j, the speed used in step j, in m/s;
- v_s and h as above, 0.158 m/s and 0.0944 s.

$$v_1 = 0.568 - 0.5 \times 0.0944 = 0.521$$

$$v_2 = 0.521 - 0.0472 = 0.474$$

$$v_9 = \max(0.158,\ 0.568 - 9 \times 0.0472)$$

$$= 0.158 \text{ m/s}$$

$$v_j = \max(v_s,\ v_a - \dot{v}_b\, j\, h)$$

The speed reaches 0.158 m/s after 9 steps, 0.85 s, and stays there; the robot covers 0.418 m in the 1.7 s and ends facing 91.82° (notebook). Figure 4 rolls out two pairs. Watch the first, keep going at 0.568 m/s: its poses run straight at the person, and pose 12, 1.02 s ahead, falls inside the dotted band where the robot's centre may not go. The second, the brake, stops short.

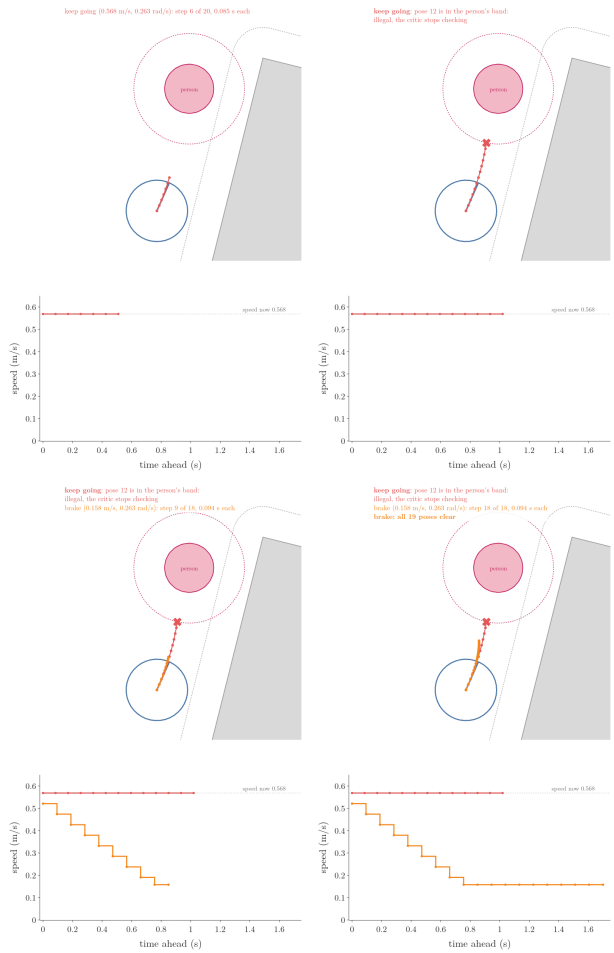


Figure 4: Two rollouts of the cycle at t = 2.4 s in our Python version of DWB's rollout, one step per frame. Red, keep going (0.568 m/s, 0.263 rad/s): 20 steps of 0.085 s; the obstacle check goes pose by pose and rejects it at pose 12, inside the dotted band round the person (the person's radius plus the robot's). Orange, brake (0.158 m/s, 0.263 rad/s): 18 steps of 0.0944 s; its speed falls by 0.0472 m/s per step and reaches 0.158 m/s after 9 steps; all 19 poses stay clear. Bottom: the speed used in each step against the time ahead

4.2 Two generators, two kinds of rollout

The two generators of Section 3.4 also roll out differently (`standard_traj_generator.cpp`, `limited_accel_generator.cpp`):

- the default generator lets the speed ramp from the measured one to the pair, as in Figure 4, because its pairs may lie up to 1.7 s of acceleration away;
- the DWA generator holds the pair from the first step, because every pair in its one-cycle window is reached within that cycle; this is Fox's assumption that the velocities after the first interval stay constant (Fox et al. 1997 §4.1).

Either way, the command sent is the pair itself, and each rollout is one constant command, never a sequence. All 375 pairs are rolled out the same way; next they must be compared.

5. Which command is best: the critics

> **Key point:** The third part of the opening problem, best: each rollout is scored by a list of critics; an obstacle critic rejects rollouts that touch an obstacle, and the rest are scored by distances to the plan and to the plan's end on a 5 cm grid, each times a weight. In the person's cycle 133 rollouts are illegal, and the lowest total, 26.0, belongs to the brake (0.158 m/s, 0.263 rad/s).

Section 4 gave 375 imagined paths; the robot needs a rule that ranks them.

The idea. Rank the rollouts by one cost made of parts with chosen weights: how far the end is from the goal, how close the path comes to obstacles, and how slow it is; the command with the lowest cost is sent. Fox's paper uses the same parts as a score to maximise, a weighted sum of the heading to the goal, the clearance and the speed (Fox et al. 1997, eq. 13), and ROS 1's DWA as a cost to minimise (Zheng 2017 §3.1.3). DWB uses a cost to minimise: each part is a separate plug-in, and the total is the sum of each part's score times its weight (`dwb_local_planner.cpp`, `scoreTrajectory`).

The standard term. A plug-in of DWB that scores one rollout for one objective, such as staying near the plan, or rejects it as illegal, is a **critic** (G-3254) (DWB README; Macenski et al. 2023 §III-A). The configuration example lists seven, in the order they run (Nav2 docs: DWB configuration): RotateToGoal, Oscillation, BaseObstacle, GoalAlign, PathAlign, PathDist and GoalDist. Most of them read the local costmap.

5.1 The costmap the critics read

A costmap, taught with costmaps in RO-043, is a grid over the floor, like an occupancy grid (G-2340), whose cells hold a cost from 0 (free) to 254 (an obstacle) for the robot's centre being there. Nav2's inflation layer sets the cost from the distance to the nearest obstacle (`inflation_layer.hpp`, `computeCost`). With:

- d, the distance from a cell's centre to the nearest obstacle, in m;
- r, the robot's radius, 0.25 m (`robot_radius`; the bring-up file's TurtleBot has 0.22 m);
- k, the `cost_scaling_factor`, 3.0 in the bring-up file (the code's default is 10.0);
- c, the cell's cost, a whole number.

The rule, by distance:

- inside an obstacle: 254;
- d up to r: 253, because a robot centred there would overlap the obstacle;
- d from r to the `inflation_radius`, 0.70 m in the bring-up file: falling with distance;
- beyond 0.70 m: 0.

$$c(0.40) = 252\, e^{-3\times(0.40-0.25)}$$

$$= 252\, e^{-0.45} = 160$$

$$c(d) = 252\, e^{-k\,(d-r)}$$

The code keeps the whole-number part. Figure 5 draws the rule and the costmap of the person's cycle. Read the top panel first: the cost is 253 out to the robot's radius, then falls. The bottom panel colours each 5 cm cell by that cost: black inside the person and the table, dark red in the band where the robot's centre may not go, orange fading to white further out.

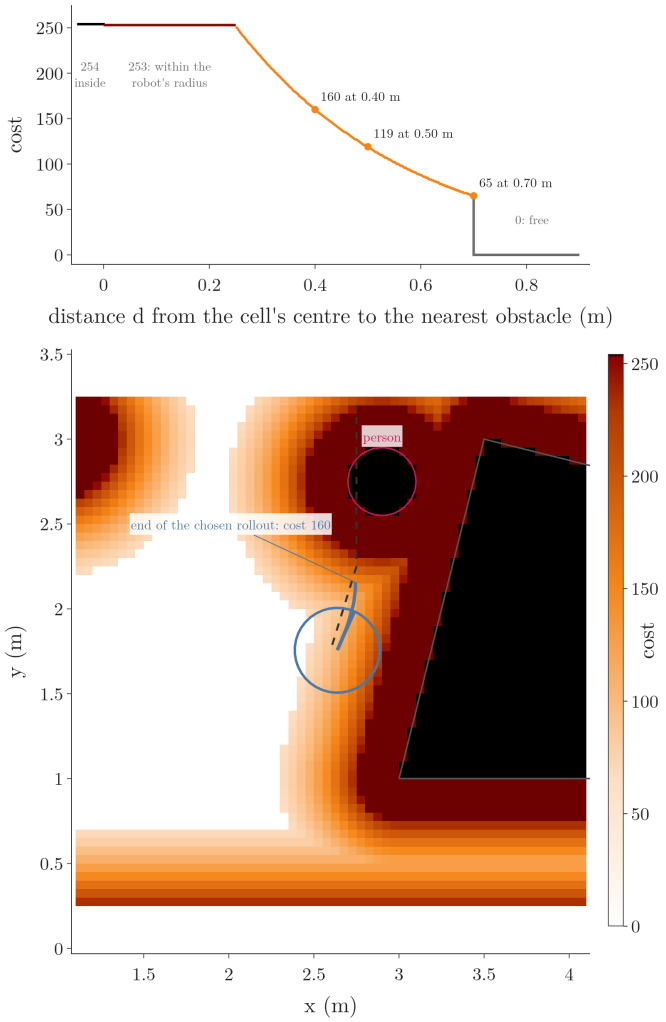


Figure 5: Top: a cell's cost against the distance d from its centre to the nearest obstacle (Nav2's inflation rule with radius 0.70 m and scaling factor 3.0, robot radius 0.25 m): 254 inside, 253 within the robot's radius, then 252 eˆ(−3(d − 0.25)): 160 at 0.40 m, 119 at 0.50 m, 65 at 0.70 m, and 0 beyond 0.70 m. Bottom: the local costmap of our Python version at t = 2.4 s, built from the room's exact shapes, 3 m by 3 m of 5 cm cells round the robot (blue), coloured by cost, with the person, the table, the plan (dashed) and the chosen rollout, whose end cell costs 160

5.2 The obstacle critic: illegal or a small cost

BaseObstacle looks up the cost of every pose of a rollout (`base_obstacle.cpp`):

- if any pose lies on a cell of cost 253 or 254, or off the costmap, the rollout is illegal and dropped;
- otherwise its score is the cost of the last pose's cell.

In the person's cycle this drops 133 of the 375 rollouts: every pair fast enough to reach the person's band or the table's within 1.7 s (notebook). The chosen rollout ends 0.40 m from the person, cost 160, and BaseObstacle's weight, its `scale`, is 0.02:

$$160 \times 0.02 = 3.2$$

For a robot that is not round, **ObstacleFootprint** checks the cells under the outline of the robot at each pose instead (`obstacle_footprint.cpp`); for a disk, BaseObstacle on a costmap inflated by the radius gives the same answer, which is why the README says BaseObstacle needs the inflation layer.

5.3 Following the plan: the distance grids

The rollouts that remain must also follow the plan and make progress along it. Four critics measure that on grids of distances, all built the same way (`map_grid.cpp`, `costmap_queue.cpp`):

1. mark the source cells with 0;
2. spread out to neighbouring cells, nearest first, through the four side neighbours;
3. give each cell its distance, in cells, from the source that reached it: steps along the rows plus steps along the columns, with no diagonals, the Manhattan distance (the Minkowski distance with $p = 1$, G-1227).

The two grids, in the person's cycle (Figure 6):

- **the plan grid:** the sources are the cells of the local plan, the 30 plan poses from the one closest to the robot to where the plan leaves the costmap, 1.43 m;
- **the goal grid:** the source is the plan's last cell on the costmap, at (2.75, 3.2).

Watch the distances spread through the person too: these grids ignore obstacles, which are BaseObstacle's job.

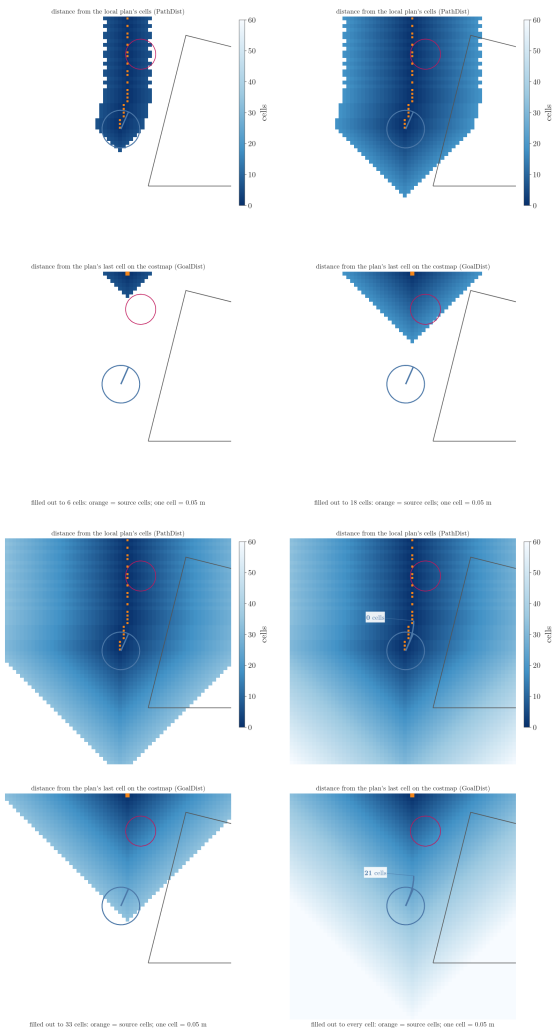


Figure 6: The two distance grids of the cycle at t = 2.4 s in our Python version, filled out from their source cells (orange) in order of distance, one frame per 3 cells; darker = nearer. Top: from every cell of the local plan (PathDist and PathAlign read it). Bottom: from the plan's last cell on the costmap (GoalDist reads it). The grids spread through the person, because obstacles are left to BaseObstacle. Last frame: the chosen rollout ends on a cell 0 cells from the plan and 21 cells from the plan's end; one cell is 0.05 m

The four critics read these grids at one point each (`path_dist.cpp`, `goal_dist.cpp`, `path_align.cpp`, `goal_align.cpp`):

- **PathDist:** the plan grid at the rollout's last pose: how far the rollout ends from the plan;
- **GoalDist:** the goal grid at the last pose: how far it still is from the plan's end, so a rollout that gets further along scores lower;
- **PathAlign:** the plan grid at a point `forward_point_distance`, 0.1 m, in front of the last pose: a rollout that ends on the plan but facing across it scores higher;
- **GoalAlign:** the same 0.1 m point, on a goal grid whose source is the plan's end pushed

0.1 m on towards the goal; when that point leaves the costmap, the last plan cell before it is used.

For the chosen rollout, with cells numbered across (i) and up (j) from the costmap's corner: its last pose is in cell (32, 38), its 0.1 m point in (32, 40), the plan's end in (32, 59), GoalAlign's source in (32, 57). Both points lie on plan cells, so PathDist and PathAlign are 0. The other two:

$$\text{GoalDist} = |32 - 32| + |59 - 38| = 21$$

$$\text{GoalAlign} = |32 - 32| + |57 - 40| = 17$$

The weights. The four grid critics count in cells. Their weight is their `scale` times the cell size times 0.5 (`map_grid.hpp`, `getScale`): the cell size turns a count of cells into metres, so a `scale` means the same at any costmap resolution. With the example's scales, 32 for PathDist and PathAlign and 24 for GoalDist and GoalAlign, and cells of 0.05 m:

$$32 \times 0.05 \times 0.5 = 0.8 \text{ per cell}$$

$$24 \times 0.05 \times 0.5 = 0.6 \text{ per cell}$$

One cell is 0.05 m, so ending 0.05 m off the plan costs 0.8, as much as a cost of 40 on BaseObstacle.

5.4 Two critics for the goal and for oscillation

Two critics of the example score 0 in the person's cycle but matter elsewhere:

- **RotateToGoal** (`rotate_to_goal.cpp`, scale 32): once the robot is within 0.25 m of the goal and less than 1.0 m of plan is left, it rejects any rollout that does not slow down; once the robot is slower than 0.25 m/s, it rejects any rollout that drives at all and scores the rest by how far their last heading is from the goal's heading. So the robot stops near the goal and then turns on the spot.
- **Oscillation** (`oscillation.cpp`): after the chosen speed, or the turn rate while standing, has changed sign once, it rejects rollouts that would change it back, until the robot has moved 0.05 m or turned 0.2 rad; this stops the robot oscillating forwards and backwards.

RotateToGoal and Oscillation shape the end of the drive (Section 7); in the person's cycle the total of the other five decides.

5.5 The total, and the lowest one

DWB adds each critic's score times its weight and keeps the rollout with the lowest total (`scoreTrajectory`, `coreScoringAlgorithm`). The chosen pair, (0.158 m/s, 0.263 rad/s), on our numbers, with the weights of Section 5.3:

$$160 \times 0.02 + 17 \times 0.6 + 0 \times 0.8$$

$$+0 \times 0.8 + 21 \times 0.6$$

$$= 3.2 + 10.2 + 0 + 0 + 12.6$$

$$= 26.0$$

In symbols, with:

- s_i, critic i's raw score of the rollout;
- w_i, its weight, 0.02 for BaseObstacle and 0.6 or 0.8 per cell for the grid critics;
- J, the rollout's total, the lower the better;
- $\sum_i$, the sum over the seven critics, as in the line above.

$$J = \sum_i w_i \, s_i$$

The fastest pairs cannot have the lowest total here. The best legal pair at 0.5 m/s or more, (0.505 m/s, 0.895 rad/s), swerves left round the person and ends off the plan:

$$200 \times 0.02 + 19 \times 0.6 + 9 \times 0.8$$

$$+6 \times 0.8 + 20 \times 0.6$$

$$= 4.0 + 11.4 + 7.2 + 4.8 + 12.0$$

$$= 39.4$$

The swerve gets 1 cell further along (GoalDist 20 against 21) but pays 12.0 for leaving the plan. So in the person's cycle the weight on staying near the plan makes braking the best answer. A second pair, (0.190 m/s, 0.263 rad/s), also totals 26.0: its rollout ends in the same

cells. The code replaces the best only when a total is strictly lower, and pairs are tried from the slowest speed up, so the slower one is kept (notebook).

Figure 7 plays the whole cycle. Watch the bottom panel fill in, one sampled speed per frame: the slow pairs (bottom rows) are all legal; from about 0.25 m/s up, the pairs that turn right run into the table's band and the straight ones into the person's (red crosses). The last frame marks the lowest total.

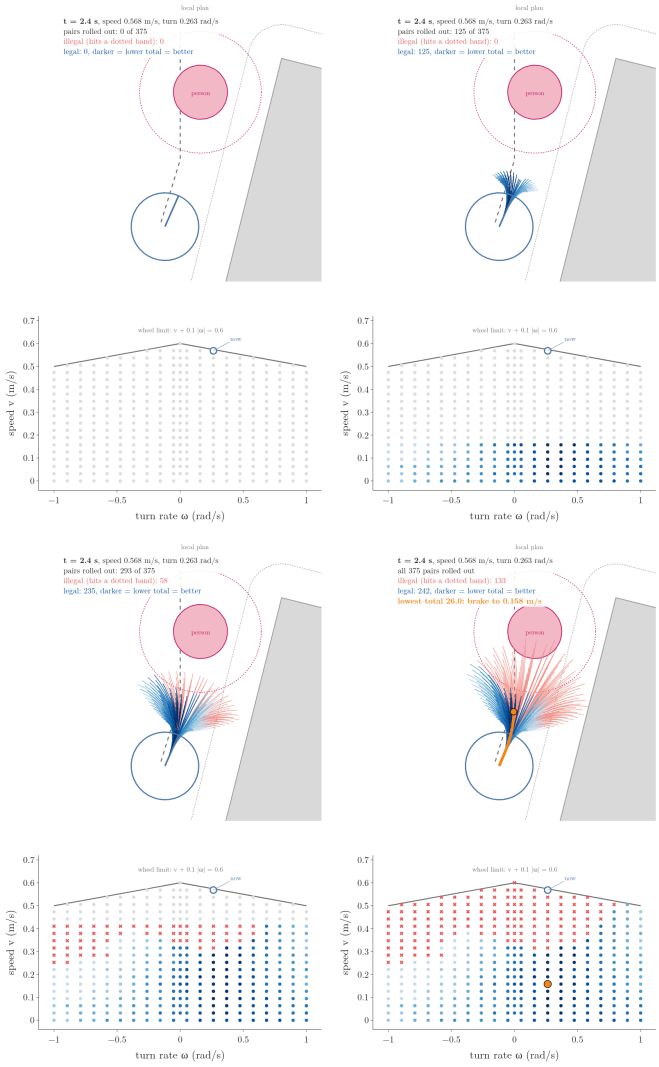


Figure 7: The cycle at t = 2.4 s, one frame per sampled speed, in our Python version of DWB's algorithm. Top: each of the 375 pairs rolled out for 1.7 s from the robot (blue circle). Red: illegal, a pose inside a dotted band (the person's or the table's radius plus the robot's). Blue: legal, darker = lower total = better. Bottom: the same pairs as points of the speed and turn-rate plane, inside the wheel limit (grey line); the blue ring is the command now. Last frame: 133 illegal, 242 legal; the lowest total, 26.0, is the brake (0.158 m/s, 0.263 rad/s), orange

The part of the opening problem called best has its answer: brake to 0.158 m/s and turn left

gently. One cycle is decided; the robot must now repeat it 20 times a second.

6. One cycle start to finish, in the order the code runs

> **Key point:** One cycle of Nav2's controller server runs 12 steps: wait for the costmap, take any new plan, read the pose, cut the plan to 2 m inside the costmap, check the goal and the progress, prepare the critics, sample, roll out and score, keep the lowest, and send it.

Sections 3 to 5 took DWB's work apart; Figure 8 puts it back in the order the code runs, at 2.4 s. The controller server is Nav2's, so its own steps are only named here and taught in Nav2 start to finish. Watch each step light up with its value.

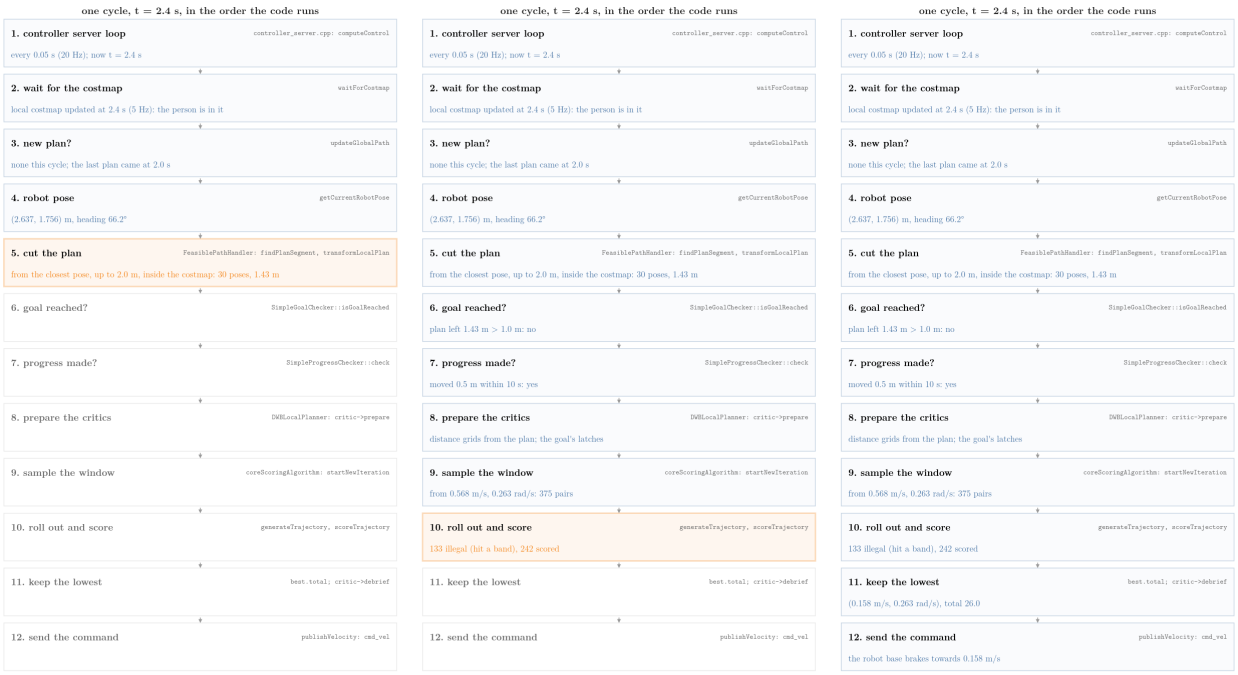


Figure 8: One cycle at t = 2.4 s in the order the navigation2 code runs it, each step with its function and the value in our Python version: 1 the controller server loop, every 0.05 s; 2 wait for the local costmap, updated at 2.4 s with the person in it; 3 no new plan this cycle (the last came at 2.0 s); 4 the robot's pose (2.637, 1.756) m, heading 66.2°; 5 cut the plan: from the closest pose, up to 2.0 m, inside the costmap, 30 poses, 1.43 m; 6 goal reached? no, 1.43 m of plan left; 7 progress made? yes; 8 prepare the critics; 9 sample the window, 375 pairs; 10 roll out and score, 133 illegal, 242 scored; 11 keep the lowest, (0.158 m/s, 0.263 rad/s), total 26.0; 12 send it on cmd_vel

The steps, with the files on `main`:

1. **The loop** (`controller_server.cpp`, `computeControl`): a 20 Hz loop, `controller_frequency` 20.0, started when the behaviour tree sends the plan with its `FollowPath` action.

2. **The costmap** (`waitForCostmap`): wait until the local costmap is current; ours was rebuilt at 2.4 s, 5 times a second.
3. **A new plan** (`updateGlobalPath`): if one arrived, hand it on; DWB's critics reset then (`newPathReceived`).
4. **The pose** (`getCurrentRobotPose`): (2.637, 1.756), facing 66.2°.
5. **The plan cut to size** (`FeasiblePathHandler`): find the plan pose closest to the robot, drop the poses before it, keep up to `prune_distance`, 2.0 m, and stop where the plan leaves the costmap: 30 poses, 1.43 m.
6. **The goal** (`SimpleGoalChecker`): reached when less than 1.0 m of plan is left, the robot is within 0.25 m of the goal and its heading within 0.25 rad; goal and progress checks are taught with the behaviour tree.
7. **Progress** (`SimpleProgressChecker`): abort if the robot has not moved 0.5 m within 10 s.
8. **The critics prepare** (`dwb_local_planner.cpp`, `prepare`): build the distance grids of Section 5.3.
9. **The window** (`startNewIteration`): the 375 pairs of Section 3.4.
10. **Roll out and score** (`generateTrajectory`, `scoreTrajectory`): Sections 4 and 5; a rollout stops being scored as soon as its running total passes the best so far, which saves time but never changes the winner (`short_circuit_trajectory_evaluation`).
11. **Keep the lowest** and let the critics note it (`debrief`): the Oscillation critic records the sign of the chosen command.
12. **Send it** (`publishVelocity`) on `cmd_vel`. If no rollout is legal, DWB raises an error; the controller server sends a stop and tries again, and aborts only after `failure_tolerance`, 0.3 s in the bring-up file.

One cycle is complete. The last part of the opening problem is whether 20 of these a second get the robot to the door.

7. The whole drive: DWB against TEB

> **Key point:** In our Python version, with the plan made once, as in RO-058, DWB stops 0.07 m in front of the person and is aborted at 12.45 s: it follows the plan and cannot leave it. With the plan remade every second, as in Nav2's released versions, it goes round and is accepted at 13.3 s, 0.23 m from the door pose, against 8.0 s for TEB.

The cycle of Section 6 chose to brake. What happens next depends on the plan the critics, DWB's scoring functions, follow.

7.1 The plan made once: stopped in front of the person

RO-058 made the global plan once, and TEB bent round the person on its own. Figure 9 runs our Python version of DWB both ways, all else equal. Watch the purple robot, whose plan is made once: it approaches the person slowly and stops.

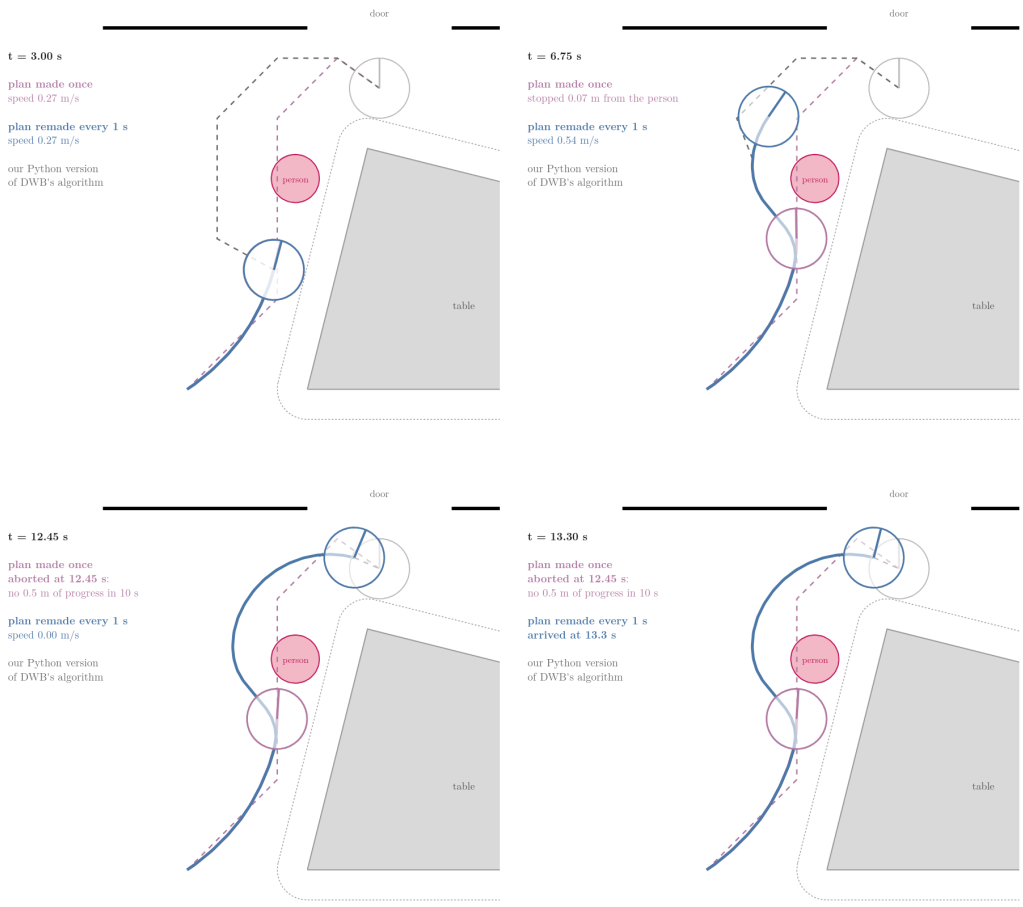


Figure 9: Two runs of our Python version of DWB's algorithm (not Nav2 running), one frame per 0.25 s. Purple: the global plan made once (purple dashed), as in RO-058; the robot brakes for the person, stands still 0.07 m from them from 6.75 s, and is aborted at 12.45 s because it has not moved 0.5 m in 10 s. Blue: the plan remade every second (grey dashed), as the default behaviour tree of Nav2's released versions does; the plan made at 3.0 s goes left of the person, and the goal checker accepts the robot at 13.3 s. The two robots drive almost together until 3.0 s (0.003 m apart)

The purple robot stands still from 6.75 s, 0.07 m from the person, and the progress checker aborts it at 12.45 s (notebook). The reason is in the weights of Section 5.5: every rollout, the path a command would drive in 1.7 s, that ends off the plan pays 0.8 per cell on PathDist alone, while the rollouts along the plan run into the person and are illegal. Of what is left, standing still costs least. DWB follows the plan it is given and never searches for another way round, so it can stop where no reachable command scores better, a local minimum (Macenski et al. 2023 §III: reactive planners are "often susceptible to local minima"). DWB follows the plan, as Nav2's documentation describes a controller, so the plan above it must change.

7.2 The plan remade every second

The default behaviour tree of Nav2's released versions (Humble, Jazzy, Kilted) remakes the plan once a second, from where the robot is, on a map that by then holds the person. The tree on `main` checks once a second and keeps the old plan while it is still valid; here the person makes it invalid, so it too plans again, at 3.02 s in the run of RO-079 (Nav2's default tree). In the blue run of Figure 9, the plan made at 3.0 s runs from the robot to (2.25, 2.25), up to (2.25, 3.25), and on to the door pose, 3.176 m: left of the person (notebook). DWB follows it:

- **3.0 s to 5.0 s:** the robot turns left and passes the person, 0.002 m from them at 4.75 s; in 6 cycles between 4.5 s and 5.0 s every rollout touches a band, and the controller server sends a stop and tries again within its `failure_tolerance` of 0.3 s;
- **5.0 s to 9.7 s:** it follows the plan to the door at up to 0.54 m/s;
- **9.7 s to 13.3 s:** within 0.25 m of the door pose, RotateToGoal holds it still while it turns from −13.85° to 75.99°;
- **13.3 s:** the goal checker accepts it: within 0.25 m of the door pose (0.23 m) and within 0.25 rad (14.3°) of its heading.

The robot passes so close because no critic keeps it further from the person: BaseObstacle only forbids cells within the robot's radius, measured at cell centres 5 cm apart, and its weight of 0.02 makes even the highest legal cost, 252, worth 5.04, about 6 cells of PathDist (notebook).

7.3 Why DWB is slower than TEB here

The DWB rows are runs of our Python version; the TEB rows are RO-058's.

Run	Arrival	At the door pose
TEB, RO-058, clear room	6.7 s	stopped on it
TEB, RO-058, person	8.0 s	stopped on it
DWB, clear room, plan made once	9.35 s	accepted 0.233 m from it
DWB, person, plan made once	aborted at 12.45 s	1.51 m away
DWB, person, plan remade every second	13.3 s	accepted 0.23 m from it

Our runs show three reasons, each from the code:

- **One command per rollout.** Each rollout holds one pair for 1.7 s, so DWB cannot plan a swerve followed by a return to the plan, and in the person's cycle every fast pair either hits or leaves the plan; it brakes to 0.27 m/s by 3.0 s. TEB optimises a whole band of poses and kept 0.44 m/s (RO-058 §1). Rösmann et al. 2021 describe DWA as a receding-horizon scheme whose control stays constant over the horizon.

- **It waits for the plan.** DWB gets round only after the plan made at 3.0 s; TEB bent its own band in the cycle the person appeared.
- **It turns at the end.** The plan's last piece reaches the door pose heading down and to the right, so DWB arrives facing about $-14°$ and turns on the spot for 3.6 s. RotateToGoal scores the heading at the end of each 1.7 s rollout, so it picks turn rates that land the heading on the goal 1.7 s later, and the turn slows as the gap closes. TEB plans every pose's heading and arrives facing 90°. Even in the clear room DWB starts its turn at 6.7 s and is accepted at 9.35 s.

Each DWB cycle, in return, is simple: no solver and no first guess, just 375 rollouts and sums. ROS has shipped this algorithm in its navigation software since ROS 1, which the next section shows.

8. Real systems: DWA in ROS 1 and DWB in Nav2

> **Key point:** ROS 1's `dwa_local_planner` scores with three weights (0.6, 0.8 and 0.01 by default); Nav2's DWB generalises it into plug-in generators and critics, was long Nav2's default controller, and is being replaced by MPPI in Nav2's bring-up parameters.

The room's controller reproduces two real packages; their defaults show how DWA has been used.

ROS 1: `dwa_local_planner` (`ros-planning/navigation`, `noetic-devel`, `cfg/DWAPlanner.cfg`). Its defaults: `sim_time` 1.7 s, `vx_samples` 3, `vth_samples` 20, `forward_point_distance` 0.325 m, `use_dwa` true (the window of one `sim_period`, as Fox's). Its cost is three weighted terms, the lowest winning (Zheng 2017 §3.1.3):

- `path_distance_bias`, 0.6 by default, times the distance from the rollout's end to the plan;
- `goal_distance_bias`, 0.8 by default, times the distance from the end to the local goal;
- `occdist_scale`, 0.01 by default, times the highest obstacle cost along the rollout.

The tuning guide's robot used 32.0, 20.0 and 0.02, close to Nav2's example weights, and 20 speed and 40 turn-rate samples (Zheng 2017 §3.1.2–3.1.3).

Nav2: `nav2_dwb_controller` (`ros-navigation/navigation2`, `main`). Its README calls DWB "the successor to the base local planner and DWA controllers in ROS 1", with plug-in trajectory generators and critics that "can be dynamically reconfigured, reweighted, and tuned". The code's defaults differ from the example in places: `linear_granularity` 0.5 m (example 0.05 m) and `forward_point_distance` 0.325 m (example 0.1 m); `sim_time` 1.7 s, `vx_samples` 20, `vy_samples` 5, `vtheta_samples` 20 and each critic's `scale` 1.0 are the same in both. The survey of Nav2's maintainers gives DWB's highest rate as 250 Hz, against 125 Hz for MPPI and 130 Hz for TEB, and warns that DWB "suffers from numerous, highly interrelated parameters" (Macenski et al. 2023 §III-A, Table II).

Default or not. Nav2's documentation still says "The DWB controller is the default controller" (Nav2 docs: DWB configuration), and the survey calls it the default "largely due to historical precedence". But in `nav2_bringup/params/nav2_params.yaml` on `main`, the

`FollowPath` controller is `nav2_mppi_controller::MPPIController`, which the maintainers planned as DWB's replacement (Macenski et al. 2023 §III-A). Nav2's other controllers include regulated pure pursuit and TEB as a third-party plug-in (TEB start to finish); MPPI, which samples whole sequences of commands rather than one constant command, has its own later Note.

9. Summary

Step	What DWB does	In the room at 2.4 s
window	keeps the commands reachable within `sim_time` at the acceleration limits	0 to 0.6 m/s, −1 to 1 rad/s
samples	20 speeds by 20 turn rates (plus 0), within the wheel limit	375 pairs
rollout	simulates each pair for 1.7 s, speed ramping at the limits	brake: 18 steps of 0.0944 s
obstacle critic	drops rollouts on cells of cost 253 or 254	133 illegal
grid critics	Manhattan cells to the plan and to its end, 0.6 or 0.8 per cell	GoalDist 21, GoalAlign 17
total	lowest weighted sum is sent	26.0: (0.158 m/s, 0.263 rad/s)
loop	20 times a second in the controller server	accepted at 13.3 s

- In our Python version of DWB, the robot reaches the door at 13.3 s with the plan remade every second, against TEB's 8.0 s, because DWB brakes in the person's cycle (every fast command hits or leaves the plan), waits for the plan of 3.0 s to go round, and turns on the spot for 3.6 s at the end; with the plan made once it stops 0.07 m from the person and is aborted at 12.45 s.
- The **dynamic window** (G-3251) holds the commands reachable in one interval, ±0.025 m/s and ±0.16 rad/s per 0.05 s cycle at 2.4 s, because speed and turn rate change only as fast as the acceleration limits allow; the **dynamic window approach** (G-3250) picks from it.
- An **admissible velocity** (G-3252) lets the robot stop before the obstacle on its arc: at 2.4 s it needs 0.323 m of the 0.578 m, so 0.568 m/s is still safe; DWB covers this with rollouts of 1.7 s, longer than the 1.2 s needed to stop.
- DWB's default **trajectory generator** (G-3256) samples the window reachable within 1.7 s, 375 pairs, because DWA's one-cycle window from rest moves the robot less than one 5 cm cell and the robot never sets off.
- A **rollout** (G-3253) simulates one constant command in small Euler steps, so the critics can judge where it goes: braking to 0.158 m/s stays clear; keeping 0.568 m/s enters the person's band at step 12.
- **Critics** (G-3254) turn each rollout into one number, because a weighted sum ranks hundreds of rollouts at once: BaseObstacle rejects or adds the end cell's cost times 0.02, the grid critics add 0.6 or 0.8 per cell from the plan and its end; the brake totals 26.0 against 39.4 for the best fast swerve.
- The **DWB controller** (G-3255) is a controller in Nav2's sense: it follows the plan it is

given and cannot search round an obstacle, so the plan above it must be remade, once a second in Nav2's default behaviour tree.

The robot can now drive to the door with the controller ROS's navigation software has shipped since ROS 1, and we have seen every step of its cycle. One question remains: what the timed elastic band package itself does in each cycle, the planner that reached the door at 8.0 s.

10. Sources

Built from

- Pranav Bhounsule, "Robotics Lec15c: Dynamic Window Approach (Fall 2024)", YouTube, https://www.youtube.com/watch?v=16TJ6NiHo6A: the teaching order, one command as one arc, the window drawn as a rectangle in the speed and turn-rate plane, the cost as a weighted sum of goal, obstacle and speed terms
- Robotics with Sakshay, "DWA Planner | Husky Robot | Motion Planning for Robots", YouTube, https://www.youtube.com/watch?v=cW_9KRL_rA0: the bundle of sampled trajectories and the chosen one in a real ROS stack, and the cycle of sampling, discarding and scoring
- Fox, D., Burgard, W. and Thrun, S. (1997). "The dynamic window approach to collision avoidance". *IEEE Robotics & Automation Magazine* 4(1), 23–33, free PDF: §4.1 search space, eq. 14 admissible velocities, eq. 15 dynamic window, eq. 13 objective (Fox et al. 1997)
- navigation2, GitHub, https://github.com/ros-navigation/navigation2 (branch main, commit 3286119, read 2026-10-12); every parameter value in this Note was read from these files:
  - nav2_dwb_controller: README; dwb_core/src/dwb_local_planner.cpp; dwb_plugins/src/standard_traj_generator.cpp, limited_accel_generator.cpp and xy_theta_iterator.cpp; dwb_plugins/include/dwb_plugins/one_d_velocity_iterator.hpp and kinematic_parameters.hpp; dwb_critics/src/ base_obstacle, obstacle_footprint, map_grid, path_dist, path_align, goal_dist, goal_align, rotate_to_goal and oscillation; costmap_queue
  - nav2_controller: src/controller_server.cpp, src/parameter_handler.cpp, plugins/feasible_path_handler.cpp, plugins/simple_goal_checker.cpp, plugins/simple_progress_checker.cpp
  - nav2_costmap_2d/include/nav2_costmap_2d/inflation_layer.hpp
  - nav2_bringup/params/nav2_params.yaml
  - nav2_system_tests/src/system/nav2_system_params.yaml
- Nav2 documentation, configuration guide, "DWB Controller", source in https://github.com/ros-navigation/docs.nav2.org (rolling): the example parameter file used for every run (Nav2 docs: DWB configuration); "Navigation Servers" concept page (Nav2 docs: navigation servers)
- Zheng, K. (2017). "ROS navigation tuning guide". arXiv 1706.09068: §3.1 DWA local planner (Zheng 2017)
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A. and Ferguson, M. (2023). "From

the desks of ROS maintainers: a survey of modern & capable mobile robotics algorithms in the Robot Operating System 2". *Robotics and Autonomous Systems* 168, arXiv 2307.15236: §III-A, Table II (Macenski et al. 2023)

Other references

- ROS navigation stack, `dwa_local_planner/cfg/DWAPlanner.cfg`, GitHub, https://github.com/ros-planning/navigation (`noetic-devel`) (DWAPlanner.cfg)
- Rösmann, C., Makarow, A. and Bertram, T. (2021). "Online motion planning based on nonlinear model predictive control with non-Euclidean rotation groups". *European Control Conference*, arXiv 2006.03534: §I (Rösmann et al. 2021)

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
DWB controller (G-3255)	Nav2's controller that, 20 times a second by default, samples speed and turn-rate commands within the acceleration limits, rolls each out, scores the rollouts with plug-in critics and sends the lowest; the successor of ROS 1's DWA, it follows the global plan and cannot search round an obstacle on its own.
Dynamic window (G-3251)	The set of speed and turn-rate commands a robot can reach from its current ones within one time interval, given its acceleration limits, such as ± 0.025 m/s and ± 0.16 rad/s per 0.05 s cycle; it moves with the current command every cycle and limits the commands worth trying.
Dynamic window approach (DWA) (G-3250)	A local planner that, every cycle, keeps only the speed and turn-rate commands the robot can reach within one interval at its acceleration limits, scores the arc each would drive, and sends the best; it avoids collisions with little computation and no solver, but chooses only among constant commands.
Admissible velocity (G-3252)	A speed and turn-rate command from which the robot, braking at its limit, can stop before the nearest obstacle on its arc, $v \le \sqrt{2\,d\,\dot{v}_b}$; DWA keeps only such commands so that it can always stop in time.

Term	Meaning
Trajectory generator (DWB) (G-3256)	The plug-in of DWB that lists the commands to try and rolls each one out: the standard generator samples everything reachable within the whole look-ahead time and ramps the speed, the limited-acceleration generator samples one cycle's window as DWA does.
Rollout (forward simulation) (G-3253)	Simulating one command forward from the robot's current pose and speed for a set time, such as 1.7 s in small steps, to get the path it would drive as a list of poses; a sampling controller scores rollouts to choose its command.
Critic (DWB) (G-3254)	A plug-in scoring function of Nav2's DWB controller that gives each rollout a score for one objective, such as ending near the plan or far from obstacles, or rejects it as illegal; DWB adds the scores times their weights and keeps the lowest total.
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.
Feedback (G-2941)	Using a measurement of what a system does to decide its next command, such as pushing harder when the measured speed is too low; it lets a controller correct errors whatever caused them.
Linear velocity v (G-2291)	How fast a robot's reference point (for a differential drive, the middle of the axle) moves along its heading, in m/s; for a differential drive, the average of the two wheel speeds, so it sets how fast the robot advances.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.

Term	Meaning
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.
Occupancy grid (G-2340)	A grid map that stores in each cell a number between 0 and 1, such as the probability that the cell is occupied, instead of a hard 0 or 1; the standard grid map of robotics, since sensor readings are uncertain.
Minkowski distance (G-1227)	A family of distances between two points, $(\sum_i \lvert x_i - y_i \rvert^p)^{1/p}$, set by the number p: $p = 2$ gives the Euclidean distance and $p = 1$ the Manhattan distance; KNN uses it to find the nearest neighbours.